



Improving Memory Efficiency for Training KANs via Meta Learning

Zhangchi Zhao, Jun Shu*, Deyu Meng, Zongben Xu

Xi'an Jiaotong University



Problem Definition and Contribution

Goal: To develop a memory-efficient training framework for KANs that reduces parameter count without sacrificing performance.

KANs: Foundation

- A KAN with a network structure of $[n_0, n_1, \dots, n_L]$ formulated as

$$\text{KAN}(\mathbf{x}; \mathcal{W}) = \Phi^{(L-1)} \circ \dots \circ \Phi^{(0)}(\mathbf{x})$$

- Each layer $\Phi^{(l)}$ is an $n_{l+1} \times n_l$ matrix of learnable activation functions:

$$\Phi^{(l)} = \begin{bmatrix} \phi_{1,1}^{(l)}(\cdot) & \dots & \phi_{1,n_l}^{(l)}(\cdot) \\ \vdots & \ddots & \vdots \\ \phi_{n_{l+1},1}^{(l)}(\cdot) & \dots & \phi_{n_{l+1},n_l}^{(l)}(\cdot) \end{bmatrix}$$

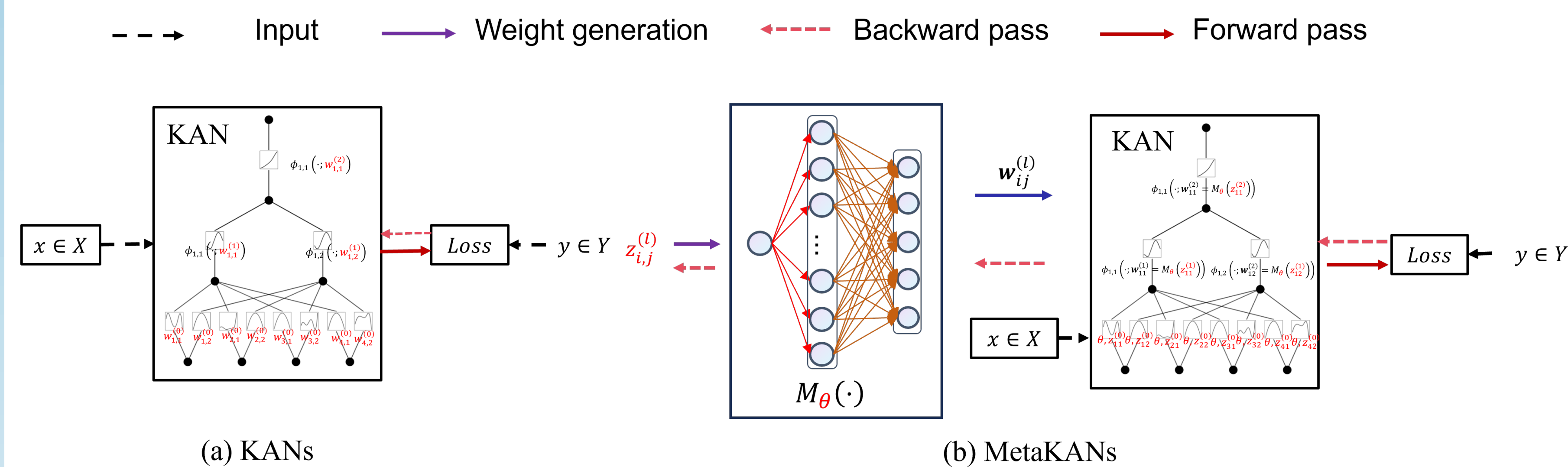
- Each function ϕ is a **learnable B-spline curve** parameterized by its weight $\mathbf{w}$: $\phi(t; \mathbf{w}) = \mathbf{w}^\top \mathbf{B}(t)$, and it leads to a parameter explosion.

Params (KAN): $\sum_{l=0}^{L-1} (n_l n_{l+1}) \times (G + k + 1)$

Params (MLP): $\sum_{l=0}^{L-1} (n_l n_{l+1})$

Insight

Key Contribution: MetaKANs, a framework that uses meta-learning to improve the memory efficiency of KANs while maintaining competitive performance on various benchmarks.



Main Idea: KAN weight learning $\rightarrow$ shared weight-generation rule.

1. KAN as a Multi-Task Problem In a KAN, learning the weight vector $\mathbf{w}$ for each spline is treated as a separate "task":

$$f(\mathbf{x}) \approx \sum_j \phi_j^{(1)} \left(\sum_i \phi_{i,j}^{(0)}(x_i; \mathbf{w}_{i,j}^{(0)}); \mathbf{w}_j^{(1)} \right), \phi \in \mathcal{F} = \{f | f(x) = \mathbf{w}^\top \mathbf{B}(x), \mathbf{w} \in \mathbb{R}^{G+k+1}\}.$$

2. Solution

All tasks follow an abstract shared rule M .

$$M: \mathcal{T} \rightarrow \mathbb{R}^{G+k+1}$$

A **meta-learner**, M_θ is used to model this rule that generates the weights $\mathbf{w}$ from a small, learnable prompt z :

$$M_\theta: \mathbb{R} \rightarrow \mathbb{R}^{G+k+1}, \quad z \mapsto \mathbf{w}. \quad (1)$$

Method

MetaKANs

1. The learnable activation function is parameterized by a prompt z and the meta-learner's weights θ

$$\phi(t; z, \theta) = M_\theta(z)^\top \mathbf{B}(t) \quad (2)$$

2. The learnable parameters consisted of a set of simple prompts $\mathcal{Z}$ (each $z \in \mathcal{Z}$ identifies its activation task) and shared meta-learner's weights θ .

$$\mathcal{Z} = \bigcup_{l=0}^{L-1} \left\{ z_\alpha^{(l)} \in \mathbb{R} \mid \alpha \in [n_l] \times [n_{l+1}] \right\} \quad (3)$$

3. Optimization Objective:

$$\mathcal{Z}^*, \theta^* = \arg \min_{\mathcal{Z}, \theta} \mathbb{E}_{\mathbf{x}} [\ell(\text{MetaKAN}(\mathbf{x}; \mathcal{Z}, \theta), f(\mathbf{x}))]$$

Scaling to Deep KANs

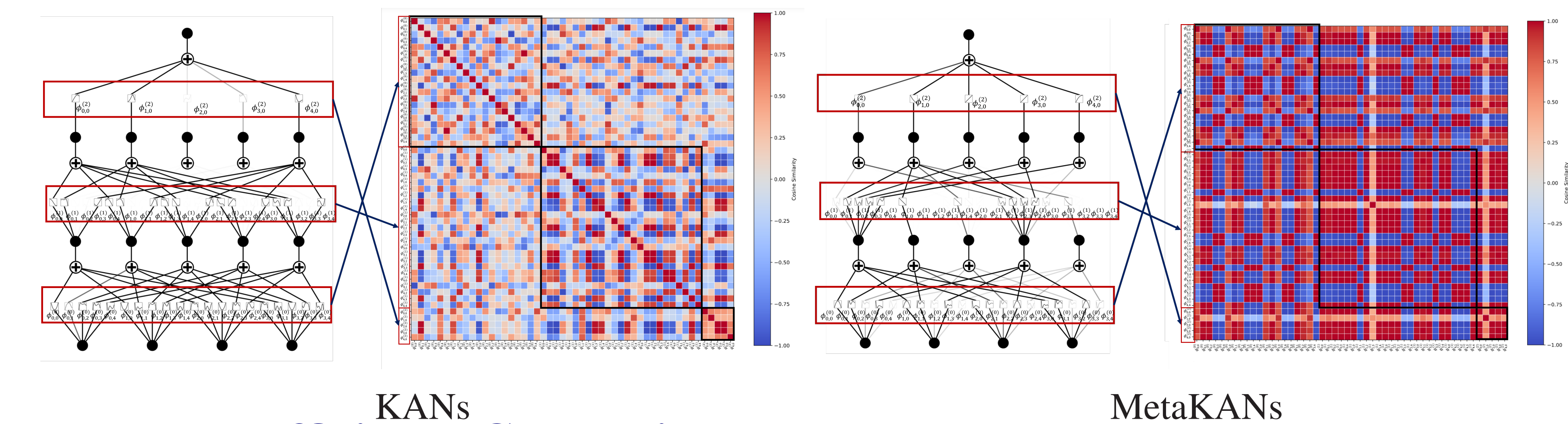
For deep KANs, the task distribution $\mathcal{F}_l = \{f | f(x^{(l)}) = \mathbf{w}^\top \mathbf{B}(x^{(l)})\}$ shifts across layers, partition L layers into C cluster sharing meta-learner $M_{(c)}$. The weight generation for a layer $l \in L_c$ is:

$$\mathbf{w}^{(l)} = M_{\theta_{(c)}}(z^{(l)})$$

Benefits

Learned Function Class Compactness

Weight similarity for function fitting task shows MetaKANs learn a more compact function space.



Parameter Efficiency Comparison

MetaKANs significantly reduce parameters across different KAN variants.

Architectures	# Params.
MLP	$\sum_{l=0}^{L-1} (n_l \times n_{l+1})$
KAN	$\sum_{l=0}^{L-1} (n_l \times n_{l+1}) \times (G + k + 1)$
MetaKAN	$\sum_{l=0}^{L-1} (n_l \times n_{l+1}) + C \times (d_{\text{hidden}} + 1) \times (G + k + 1)$
FastKAN	$\sum_{l=0}^{L-1} (n_l \times n_{l+1}) \times c$
MetaFastKAN	$\sum_{l=0}^{L-1} (n_l \times n_{l+1}) + (d_{\text{hidden}} + 1) \times c$
WavKAN	$\sum_{l=0}^{L-1} (n_l \times n_{l+1}) \times 3$
MetaWavKAN	$\sum_{l=0}^{L-1} (n_l \times n_{l+1}) + (d_{\text{hidden}} + 1) \times 3$

Experiments & Results

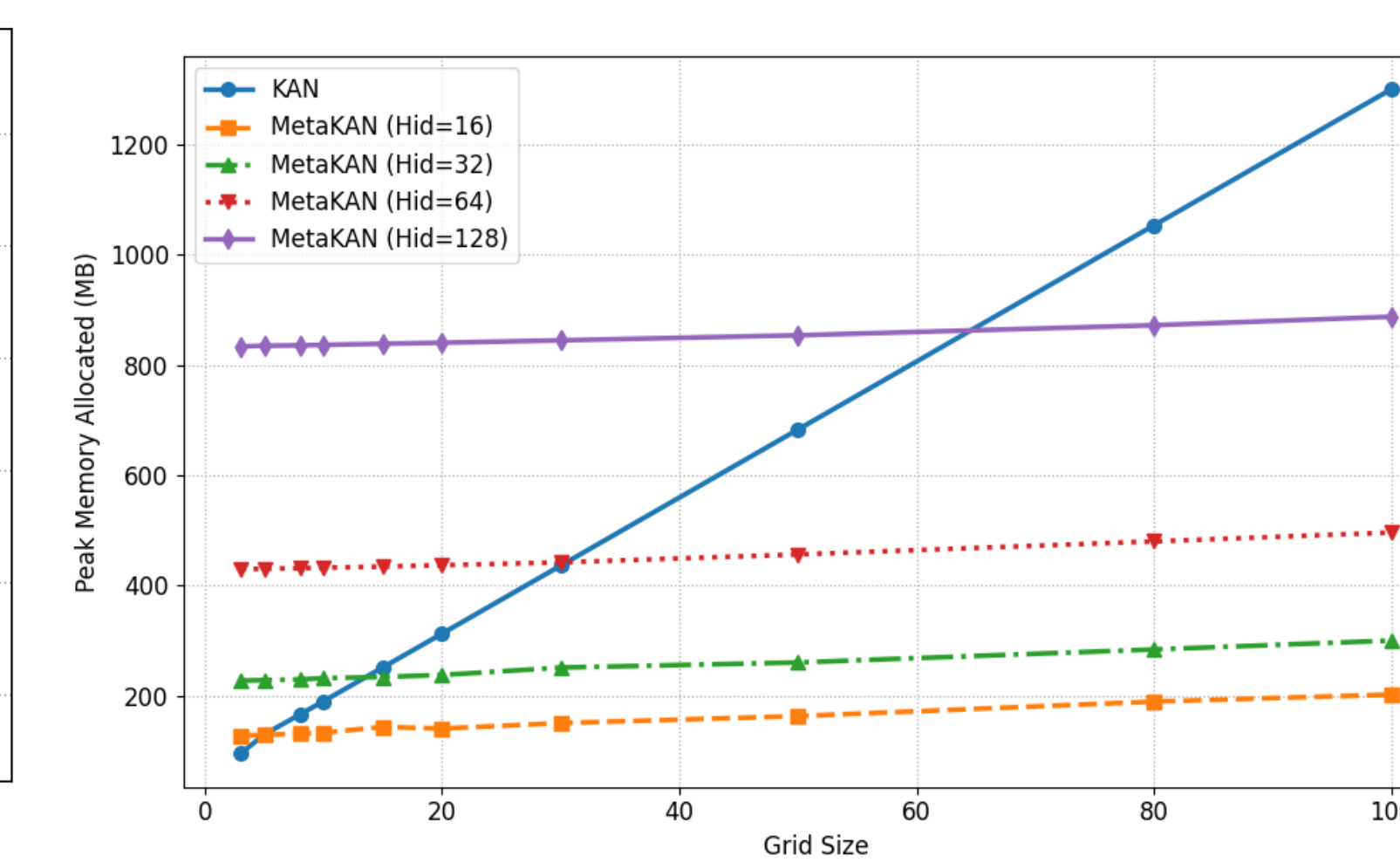
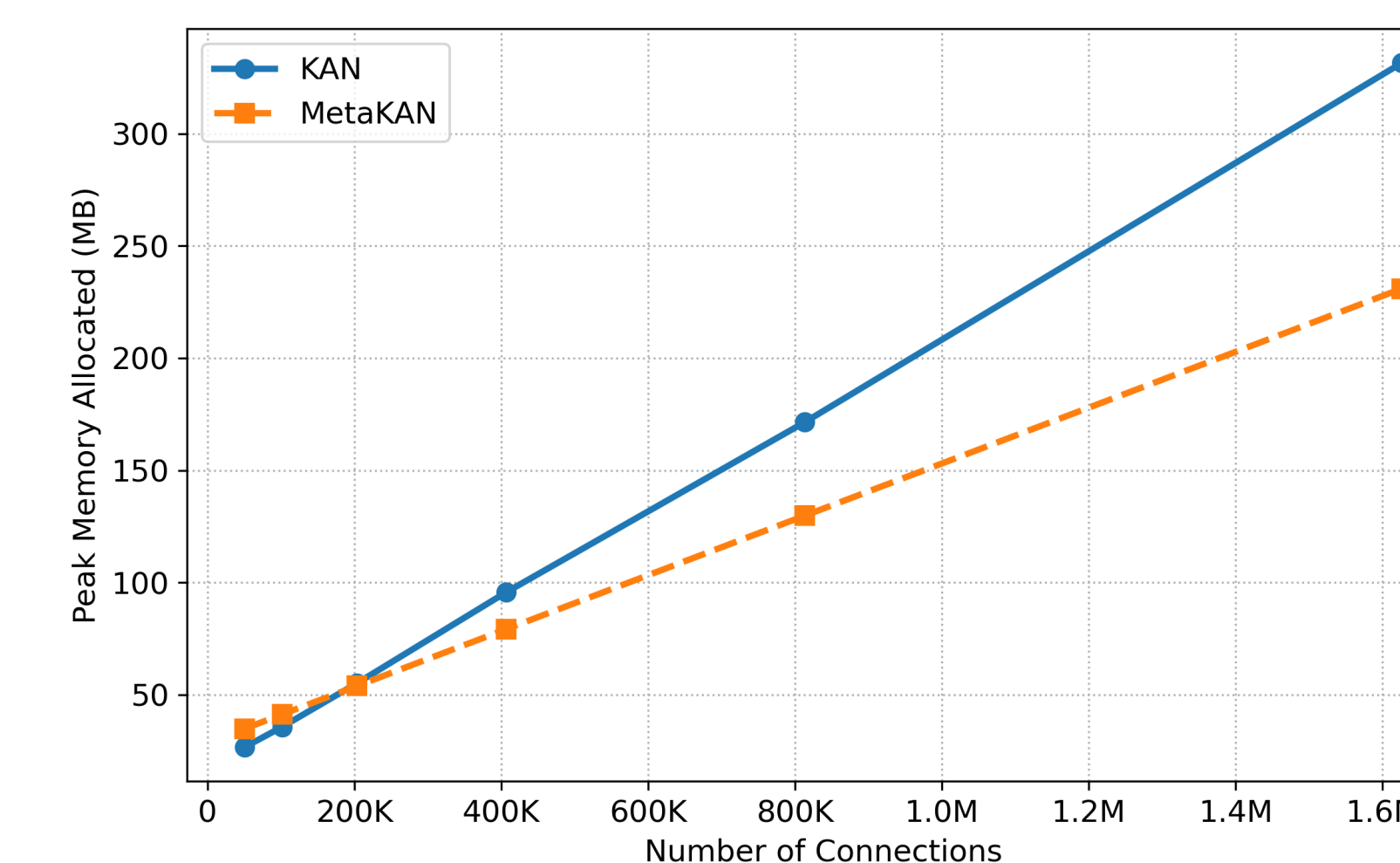
Name	Structure	$G = 5$				$G = 20$			
		KANs		MetaKANs		KANs		MetaKANs	
		MSE	# Param	MSE	# Param	MSE	# Param	MSE	# Param
1.6.20a	[1,2,1,1]	5.94×10^{-4}	58	3.88×10^{-4}	218	1.01×10^{-2}	134	5.48×10^{-3}	653
1.6.20	[2,2,1,1]	6.44×10^{-3}	76	2.67×10^{-3}	210	6.00×10^{-3}	193	4.15×10^{-4}	439
1.6.20b	[3,5,5,5,1]	4.16×10^{-3}	646	3.68×10^{-3}	1083	3.79×10^{-2}	1976	3.04×10^{-2}	1,798
1.8.4	[4,5,5,1]	1.30×10^{-2}	979	3.76×10^{-3}	461	1.16×10^{-1}	1,461	8.33×10^{-2}	1,994
1.9.18	[6,5,5,5,1]	2.39×10^{-3}	781	1.70×10^{-3}	993	1.62×10^{-2}	2,556	4.00×10^{-3}	2,029
1.12.2	[4,3,3,1]	2.83×10^{-3}	223	1.84×10^{-3}	480	1.19×10^{-2}	751	2.03×10^{-3}	672
1.12.4	[3,3,2,1]	1.87×10^{-3}	70	1.04×10^{-3}	159	4.55×10^{-3}	550	6.13×10^{-4}	665
1.12.5	[2,2,1]	1.32×10^{-3}	57	1.16×10^{-4}	30	2.97×10^{-2}	201	2.24×10^{-2}	330
1.10.7	[3,4,4,1,1]	6.96×10^{-3}	307	5.98×10^{-3}	115	9.77×10^{-3}	1,132	4.00×10^{-3}	1,329
1.12.11	[5,2,2,1]	8.73×10^{-2}	149	8.55×10^{-2}	85	2.58×10^{-1}	565	1.35×10^{-1}	1,312
1.13.4	[4,2,1,1]	4.22×10^{-3}	103	3.52×10^{-3}	55	1.84×10^{-1}	400	1.21×10^{-1}	1,955
1.13.12	[5,4,4,1]	3.41×10^{-3}	369	2.83×10^{-3}	100	1.77×10^{-2}	1,489	3.09×10^{-3}	1,336
1.14.3	[3,2,1]	4.53×10^{-3}	75	4.01×10^{-3}	40	4.66×10^{-3}	307	5.73×10^{-3}	332
1.14.4	[2,2,1,1]	1.27×10^{-3}	67	2.08×10^{-4}	35	1.16×10^{-2}	277	1.69×10^{-2}	319
1.15.3x	[4,3,3,1]	1.57×10^{-2}	223	5.38×10^{-3}	80	2.59×10^{-1}	967	2.33×10^{-1}	181
1.15.10	[3,3,2,1]	8.45×10^{-3}	159	7.79×10^{-3}	70	7.93×10^{-3}	703	8.38×10^{-3}	1,313
1.18.4	[4,4,3,1]	2.21×10^{-3}	287	1.55×10^{-3}	90	4.63×10^{-3}	1,310	8.19×10^{-3}	189

Model	MNIST		CIFAR-10		CIFAR-100	
	# Param	Acc.	# Param	Acc.	# Param	Acc.
KANConv	40,694,018	99.50 $\pm$ 0.08	40,696,610	67.24 $\pm$ 3.41	40,742,780	35.97 $\pm$ 2.84
MetaKANConv	4,532,170	99.46 $\pm$ 0.08	9,053,531	72.20 $\pm$ 2.97	9,099,371	44.17 $\pm$ 3.19
FastKANConv	40,693,052	99.71 $\pm$ 0.02	40,695,648	79.84 $\pm$ 0.46	40,741,818	50.80 $\pm$ 0.17
MetaFastKANConv	4,539,652	99.42 $\pm$ 0.08	9,051,033	78.73 $\pm$ 1.59	18,142,821	52.02 $\pm$ 1.07
KALNConv	22,611,642	69.02 $\pm$ 10.54	22,613,082	29.14 $\pm$ 1.67	22,659,252	9.10 $\pm$ 0.41
MetaKALNConv	4,529,727	99.53 $\pm$ 0.05	4,530,015	70.23 $\pm$ 2.43	4,576,185	21.94 $\pm$ 5.69
KAGNConv	22,611,674	99.58 $\pm$ 0.06	22,613,114	84.69 $\pm$ 0.74	22,659,284	56.41 $\pm$ 0.32
MetaKAGNConv	4,543,682	99.46 $\pm$ 0.20	4,530,495	84.25 $\pm$ 0.36	4,576,665	57.76 $\pm$ 0.50

Ablation study Performance consistently improves with more meta-learners (C) and larger prompt dimensions ('dim z ')

dim z	MetaFastKAN							
	C=1		C=3		C=5		C=7	
	ACC	#Param	ACC	#Param	ACC	#Param	ACC	#Param
1	40.44 $\pm$ 7.68	4,575,139	37.56 $\pm$ 1.48	4,575,861	46.05 $\pm$ 0.41	4,576,583	44.16 $\pm$ 1.01	4,576,073
2	38.62 $\pm$ 1.93	9,096,051	40.03 $\pm$ 3.46	9,096,837	46.10 $\pm$ 1.85	9,097,623	49.23 $\pm$ 2.23	9,098,409
4	42.09 $\pm$ 6.33	18,137,875	42.38 $\pm$ 1.53	18,138,789	49.03 $\pm$ 1.52	18,139,703	52.02 $\pm$ 1.07	18,140,617

Benefit to memory usage Peak GPU memory usage is lower for MetaKANs, especially for models with more connections or larger grid sizes.



Author Wechat:



Project Webpage:



Please contact:

- murphyzc0518@stu.xjtu.edu.cn
- junshu@mail.xjtu.edu.cn